

Software Design

Software Design is the process to transform the user requirements into some suitable form, which helps the programmer in software coding and implementation.

The aim of this phase is to transform the SRS document into the design document.

Objectives of Software Design

Following are the purposes of Software design:



Design concepts

The set of fundamental software design concepts are as follows:

1. Abstraction

A solution is stated in large terms using the language of the problem environment at the highest-level abstraction.

The lower level of abstraction provides a more detail description of the solution.

A sequence of instruction that contain a specific and limited function refers in a procedural abstraction.

2. Architecture

The complete structure of the software is known as software architecture.

Structure provides conceptual integrity for a system in a number of ways.

The architecture is the structure of program modules where they interact with each other in a specialized way.

The aim of the software design is to obtain an architectural framework of a system.

The more detailed design activities are conducted from the framework.

3. Patterns

A design pattern describes a design structure and that structure solves a particular design problem in a specified content.

4. Modularity

Process of dividing software components into manageable units.

This makes a program to be managed easily.

5. Information hiding

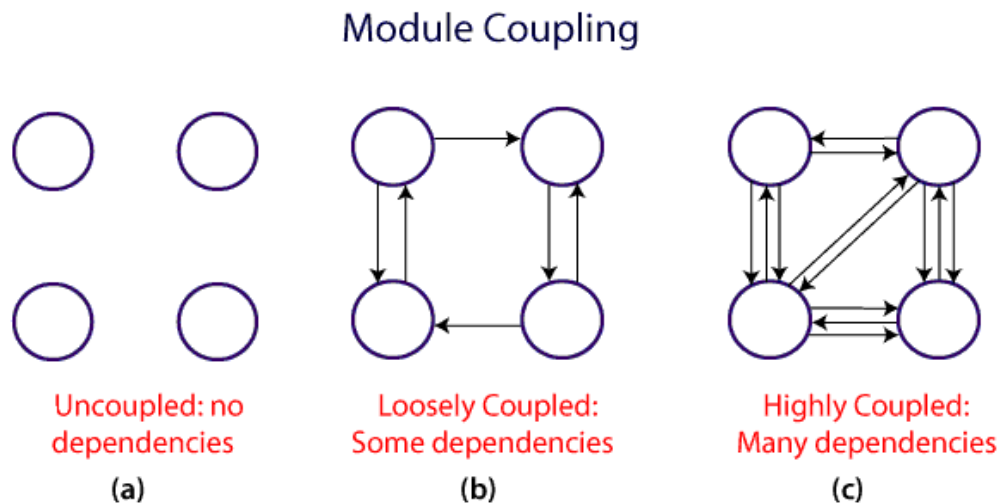
Modules must be specified and designed so that the information like algorithm and data presented in a module is not accessible for other modules not requiring that information.

6. Functional independence

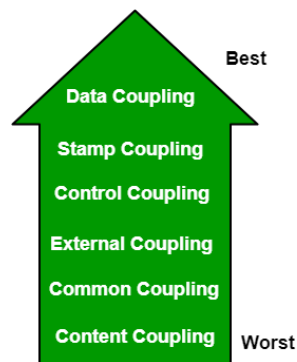
Function is atomic in nature so that it performs only a single task of the software with least interaction with other modules.

It is accessed using two criteria: 1. Cohesion and 2. coupling.

Coupling refers to degree of interdependency between software modules.
A good design is the one that has low coupling.



Types of coupling



Data coupling: Two modules are said to be 'data coupled' if they they communicate by passing only data.

Stamp coupling: Two modules are said to be 'stamp coupled' if they communicate by passing a data structure

Control coupling: If the modules communicate by passing control information, then they are said to be control coupled.

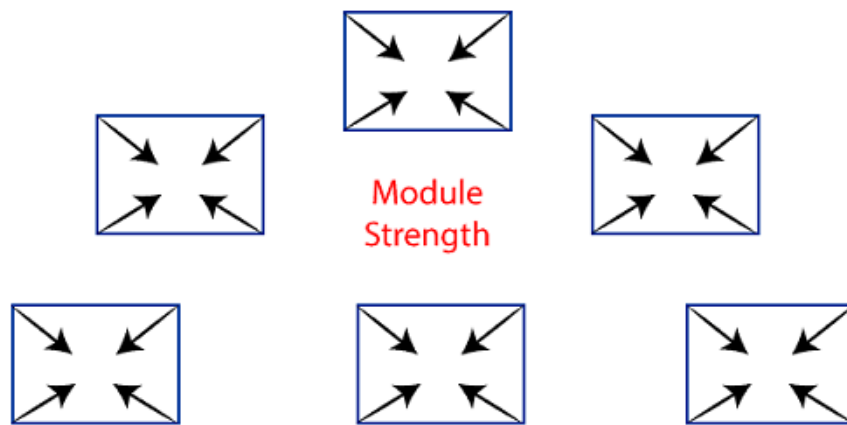
External Coupling: In external coupling, the modules depend on other modules, external to the software being developed.

Common Coupling: The modules have shared data such as global data structures.

Content Coupling: In a content coupling, one module can modify the data of another module. This is the worst form of coupling and should be avoided.

Cohesion

Cohesion refers to degree of interdependency **among** software modules.



Cohesion= Strength of relations within Modules

A good design is the one that has high cohesion.

Types of Cohesion:

Functional Cohesion: Elements within a module are grouped based on a single, specific functionality.

Sequential Cohesion: Elements are organized in a linear sequence, where the output of one element becomes the input of the next.

Communicational Cohesion: Elements within a module work together to manipulate a shared data structure.

Procedural Cohesion: Elements are grouped based on their involvement in a specific sequence of actions or steps.

Temporal Cohesion: Elements are grouped because they need to be executed at the same time

Logical Cohesion: The elements are logically related

Coincidental Cohesion: Elements have no conceptual relationship. It is the worst form of cohesion.

7. Refinement

Refinement is a top-down design approach. It is a process of elaboration.

A hierarchy is established by decomposing a statement of function in a stepwise manner till the programming language statement are reached.

8. Refactoring

It is a reorganization technique which simplifies the design of components without changing its function behaviour.

Refactoring is the process of changing the software system in a way that it does not change the external behaviour of the code still improves its internal structure.

9. Design classes

The model of software is defined as a set of design classes.

Every class describes the elements of problem domain and that focus on features of the problem which are user visible.

Software Design Principles

Design principles are used for effectively handling the complexity of the design process.

It reduces the chance of errors during design.

1. Problem Partitioning

Here, the goal is to divide the problem into manageable pieces. Thus the software becomes easy to understand, test, modify and maintain.

2. Abstraction

Abstraction reduces the complexity by hiding internal details of implementation.

Here, there are two common abstraction mechanisms

Functional Abstraction - Details of the algorithm are not visible to the user of the function

Data Abstraction- Details of the data elements are not visible to the users of data.

3. Modularity

Modularity is the process of dividing a software into manageable unit.

Each module is a well-defined and has single specified objectives.

It is measured using two criteria:

Cohesion: It measures the relative interdependence between modules.

Coupling: It measures the relative interdependence among modules.

Quality attributes

The attributes of design name as 'FURPS' are as follows:

Functionality:

It evaluates the feature set and capabilities of the program.

Usability:

It is accessed by considering the factors such as human factor, overall aesthetics, consistency and documentation.

Reliability:

It is evaluated by measuring parameters like frequency and security of failure, mean-time-to-failure (MTTF), recovery from failure etc.

Performance:

It is measured by considering processing speed, response time, resource consumption, throughput and efficiency.

Supportability:

It combines the ability to extend the program, adaptability, serviceability.

It also consists of more attributes like compatibility, extensibility, fault tolerance, reusability, robustness, security, portability.

QUALITY GUIDELINES

- A design should use recognizable architectural styles or patterns.
- Design should be modular
 - Should contain distinct representation of data, architecture & interfaces.
- Appropriate data structures for the classes to be implemented
- Independent functional characteristics for components
- Should have interfaces that reduces complexity.
- Should be derived using repeatable method

SOFTWARE ARCHITECTURE

The software architecture is the **structure** of the system, which comprise software components, the externally visible properties and the relationship among them.

It is a representation that enables a software engineer to

- *Analyze the effectiveness of the design.*
- *Considering architectural alternative when making design changes.*
- *Reduces the risk associated with the construction of the software.*

Why Is Architecture Important?

Three key reasons –

- *It enables communication and understanding between stakeholders.*
- *Highlights early design decisions to create an operational entity.*
- *Constitutes a model of software components and their interconnection.*

ARCHITECTURAL STYLES

An architectural style is a **transformation** that is imposed on the design of an entire system. This includes:

- ❖ A set of components that will perform a function required by the system.
- ❖ The set of connectors will help in coordination, communication, and cooperation between the components.
- ❖ Conditions that how components can be integrated to form the system.
- ❖ Semantic models that help the designer to understand the overall properties of the system.

1. Data centered architectures:

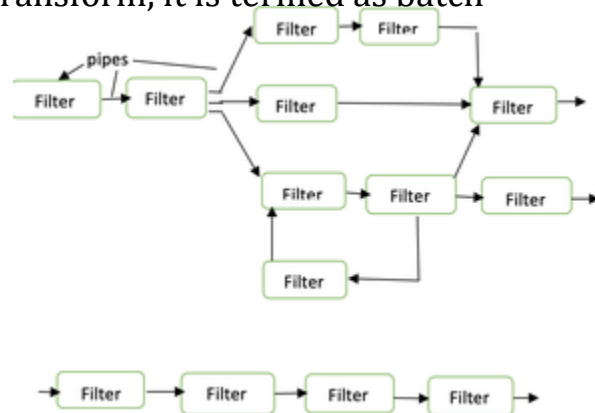
A data store will reside at the center of this architecture and is accessed frequently by the other components that update, add, delete or modify the data present within the store.



2. Data-flow architectures

It shows the flow of input data, its computational components and output data.

Structure is also called pipe and Filter. Pipe provides path for flow of data. Filters manipulate data and work independent of its neighboring filter. If data flow degenerates into a single line of transform, it is termed as batch sequential.



3. Call and return architectures

This creates a structure that is easy to modify and scale. Two sub styles:

(1) *Main program/sub program architecture* -- Classic program structure

-- Main program invokes a number of components, which in turn invoke other components

(2) *Remote procedure call architecture* -- Components of main program/subprogram are distributed across computers over network.

4. Object Oriented architecture:

The components of a system encapsulate data and the operations that must be applied to manipulate the data. The coordination and communication between the components are established via the message passing.

5. Layered architectures

A number of different layers are defined.

- Inner Layer – perform OS interfacing.
- Intermediate Layer – provide Utility services and application function.
- Outer Layer -perform User Interface operations.

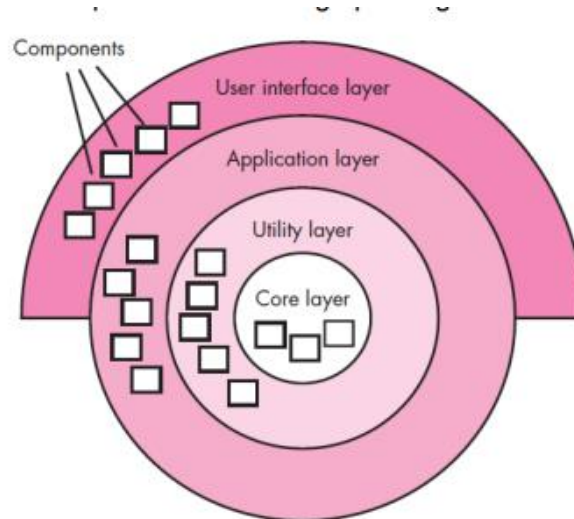


Fig 9.4: Layered architecture

Component-Level Design

After architectural design is complete, component-level design takes place.

The objective of this design is to **transform the design model into functional software.**

For this, the component-level design represents -the internal data structures and processing details of all the software components at an abstraction level. In addition, it specifies an interface that may be used to access the functionality of all the software components.

A component (also known as module) can be defined as a modular building block for the software.

The component-level design can be represented by using different approaches. For Example, use a programming language / use design notation like DFD, flowchart, or structure chart / use tabular or text-based.

Three important **views** of component are:

1. Object-Oriented View

Here, a component contains a set of collaborating classes. Each class include all attributes and operations that are relevant to its implementation.

2. Traditional View

A traditional component, also called a module, resides within the software architecture and serves one of three important roles: (1) a control component
(2) a problem domain component
(3) an infrastructure component

3. Process-Related View

This make use of existing software components or design patterns. For this, a catalog of proven design components is made available.

Class based component design and principles

Four basic design principles are applicable to component-level design.

1. **The Open-Closed Principle (OCP).** “A module should be open for extension but closed for modification”.

We should specify the component in a way that allows it to be extended without the need to make internal modifications to the component itself. To accomplish this, we create abstractions.

2. **The Liskov Substitution Principle (LSP).** “Subclasses should be substitutable for their base classes”.

This design principle, originally proposed by Barbara Liskov, suggests that a component that uses a base class should continue to function properly if a class derived from the base class is passed to the component instead.

3. **Dependency Inversion Principle (DIP).** “Depend on abstractions. Do not depend on concretions”

The more a component depends on other concrete components (rather than on abstractions such as an interface), the more difficult it will be to extend.

4. **The Interface Segregation Principle (ISP).** “Many client-specific interfaces are better than one general purpose interface”.

Interface design

User interface design creates an effective communication medium between a human and a computer.

User interface is the front-end application view to which user interacts in order to use the software.

The software becomes more popular if its user interface is:

- Attractive
- Simple to use
- Responsive in short time
- Clear to understand
- Consistent

There are two main types of User Interface:

1. Command Line Interface (CLI):

CLI provides a command prompt, the place where the user types the command using keyboard. A typical example of this is UNIX.

The user needs to remember the syntax of command and its use.

CLI Elements include: **Command Prompt, Cursor, Command.**

2. Graphical User Interface (GUI):

GUI depends much more heavily on the mouse. A typical example of this is Windows OS.

GUI Elements include: **Window, Tabs, Menu, Icon, Cursor.**

Typical design errors

1. lack of consistency
2. Too much memorization

3. No guidance / help
4. No context sensitivity
5. Poor response
6. Unfriendly

The Golden Rules

1. Place the user in control

The interaction should be defined in such a way that the user is not forced to implement unnecessary actions.

The technical internal details must be hidden from the casual user.

Design for the direct interaction with objects that appear on the screen.

2. Reduce the user's memory load

The user interface must be designed in such a way that it reduces the demands on the user's short-term memory.

Create the meaningful defaults value as an advantage for the average users in the start of application.

There must be a reset option for obtaining the default values.

The shortcut should be easily remembered by the users.

The interface screen should be friendly to users.

3. Make the interface consistent

The system must allow the user to put task into meaningful context.

Consistency should be maintained for all the interaction.

Do not change the past system that is created by the user expectation unless there is a good reason to do that.

User Interface Design Models

A software engineer establishes a user model,
the software engineer creates a design model,
the end user develops a mental image that is often called the user's mental model or the system perception, and
the implementers of the system create an implementation model.

The **user model** establishes the profile of end users of the system. To build an effective user interface, “all **design** should begin with an understanding of the intended users, including profiles of their age, gender, physical abilities, education, cultural background, motivation and personality”

The user's **mental model** is the image of the system that end users carry in their heads. For example, if the user of a particular word processor were asked to describe its operation, the system perception would guide the response. The accuracy of the description will depend upon the user's profile and overall familiarity.

The **implementation model** combines the outward manifestation of the computer-based system (the look and feel of the interface), coupled with all supporting information (books, manuals, videotapes, help files) that describes interface syntax and semantics.